

Software Engineering

Product Requirements Document

TakboHub

Technical Implementation Specification

Field	Value
Version	1.0
Date	January 2026
Type	Software Engineering PRD
Stack	React + NestJS + PostgreSQL

Table of Contents

Project Overview

Summary

TakboHub is a full-stack web application for managing running events in the Philippines. The system consists of a React SPA frontend, NestJS REST API backend, PostgreSQL database, and Redis for caching/queues. The architecture prioritizes developer experience, type safety, and scalability.

Technical Goals

- Type-safe end-to-end development with TypeScript and shared schemas
- Modular, testable architecture following SOLID principles
- Optimized for mobile-first responsive design
- Secure authentication with JWT and refresh token rotation
- Reliable payment processing with webhook handling and idempotency
- Efficient data fetching with caching and optimistic updates
- Docker-based local development matching production environment

Non-Goals (MVP)

- Native mobile applications (React Native / Flutter)
- Real-time features (WebSockets for live updates)
- Multi-tenancy with isolated databases
- Internationalization (i18n) - English only for MVP
- Offline-first / PWA capabilities

System Architecture

High-Level Architecture

The system follows a client-server architecture with clear separation between the frontend SPA and backend API. All communication happens over HTTPS with JSON payloads.

Layer	Technology	Port	Description
Frontend	React + Vite	5173	Single Page Application
API Gateway	NestJS	3000	REST API with JWT auth
Database	PostgreSQL	5432	Primary data store
Cache/Queue	Redis	6379	Sessions, jobs, rate limiting
File Storage	MinIO	9000	S3-compatible object storage
Mail (Dev)	Mailpit	8025	Local email testing

Request Flow

1. User interacts with React frontend
2. Frontend makes API request via Axios with JWT in Authorization header
3. NestJS validates JWT, extracts user context
4. Controller validates request DTO using class-validator
5. Service layer executes business logic
6. Prisma ORM queries PostgreSQL
7. Response transformed and returned to frontend
8. TanStack Query caches response, updates UI

Authentication Flow

Step	Action	Token
1	User submits login credentials	None
2	Server validates, returns tokens	Access + Refresh
3	Frontend stores refresh token (httpOnly cookie)	Refresh in cookie
4	Frontend stores access token (memory)	Access in memory
5	API requests include access token	Authorization: Bearer
6	On 401, frontend calls /refresh endpoint	Refresh cookie sent
7	Server rotates refresh token, returns new access	New tokens

Technology Stack

Frontend Dependencies

Package	Version	Purpose
react	^19.0.0	UI library with concurrent features
vite	^6.0.0	Build tool and dev server
@tanstack/react-router	^1.95.0	Type-safe file-based routing
@tanstack/react-query	^5.62.0	Server state management
zustand	^5.0.0	Client state management
axios	^1.7.0	HTTP client with interceptors
react-hook-form	^7.54.0	Form state management
zod	^3.24.0	Schema validation
@hookform/resolvers	^3.9.0	Zod integration for RHF
tailwindcss	^4.0.0	Utility-first CSS
class-variance-authority	^0.7.0	Component variants
clsx	^2.1.0	Conditional classnames
tailwind-merge	^2.6.0	Merge Tailwind classes
lucide-react	^0.469.0	Icon library
date-fns	^3.6.0	Date manipulation
sonner	^1.7.0	Toast notifications

Backend Dependencies

Package	Version	Purpose
@nestjs/core	^11.0.0	Core framework
@nestjs/platform-express	^11.0.0	Express HTTP adapter
@nestjs/config	^3.3.0	Configuration management
@nestjs/jwt	^10.2.0	JWT utilities
@nestjs/passport	^10.0.0	Authentication strategies
@nestjs/swagger	^8.1.0	OpenAPI documentation
@nestjs/throttler	^6.3.0	Rate limiting

@nestjs/bullmq	^10.2.0	Job queue integration
@nestjs/cache-manager	^2.3.0	Caching abstraction
prisma	^6.2.0	Database toolkit
@prisma/client	^6.2.0	Type-safe database client
class-validator	^0.14.0	DTO validation decorators
class-transformer	^0.5.1	Object transformation
bcrypt	^5.1.0	Password hashing
passport-jwt	^4.0.1	JWT passport strategy
nestjs-pino	^4.2.0	Structured logging
pino-pretty	^13.0.0	Log formatting (dev)

Development Dependencies

Package	Purpose
typescript ^5.7.0	Type checking and compilation
eslint ^9.0.0	Code linting with flat config
prettier ^3.4.0	Code formatting
vitest ^2.1.0	Unit testing (frontend)
@testing-library/react ^16.0.0	React component testing
jest ^29.7.0	Unit testing (backend)
supertest ^7.0.0	E2E API testing
@faker-js/faker ^9.3.0	Test data generation
husky ^9.1.0	Git hooks
lint-staged ^15.2.0	Pre-commit linting

Infrastructure

Service	Image	Purpose
PostgreSQL	postgres:16-alpine	Primary database
Redis	redis:7-alpine	Cache, sessions, queues
MinIO	minio/minio:latest	S3-compatible file storage
Mailpit	axllent/mailpit:latest	Local email testing

Database Schema

Entity Relationship Overview

The database uses PostgreSQL with Prisma ORM. All tables use UUID primary keys and include createdAt/updatedAt timestamps. Soft deletes are implemented where necessary using deletedAt.

Users Table

Column	Type	Constraints	Description
id	UUID	PK, DEFAULT uuid_generate_v4()	Primary key
email	VARCHAR(255)	UNIQUE, NOT NULL	Login email
passwordHash	VARCHAR(255)	NOT NULL	Bcrypt hash
name	VARCHAR(255)	NOT NULL	Display name
role	ENUM	NOT NULL, DEFAULT 'RUNNER'	User role
emailVerified	BOOLEAN	DEFAULT false	Email verification status
emailVerifyToken	VARCHAR(255)	NULLABLE	Verification token
passwordResetToken	VARCHAR(255)	NULLABLE	Reset token
passwordResetExpires	TIMESTAMP	NULLABLE	Token expiry
createdAt	TIMESTAMP	DEFAULT NOW()	Creation time
updatedAt	TIMESTAMP	ON UPDATE NOW()	Last update

Role ENUM values: SUPER_ADMIN, ORGANIZER, STAFF, RUNNER

Organizations Table

Column	Type	Constraints	Description
id	UUID	PK	Primary key
name	VARCHAR(255)	NOT NULL	Organization name
slug	VARCHAR(255)	UNIQUE, NOT NULL	URL-friendly identifier
ownerId	UUID	FK → users.id	Owner user reference
logoUrl	VARCHAR(500)	NULLABLE	Logo image URL

createdAt	TIMESTAMP	DEFAULT NOW()	Creation time
updatedAt	TIMESTAMP	ON UPDATE NOW()	Last update

Events Table

Column	Type	Constraints	Description
id	UUID	PK	Primary key
organizationId	UUID	FK → organizations.id	Parent org
name	VARCHAR(255)	NOT NULL	Event name
slug	VARCHAR(255)	UNIQUE, NOT NULL	URL identifier
description	TEXT	NULLABLE	Event description
date	TIMESTAMP	NOT NULL	Event date/time
location	VARCHAR(500)	NOT NULL	Venue address
locationMapUrl	VARCHAR(500)	NULLABLE	Google Maps link
bannerUrl	VARCHAR(500)	NULLABLE	Banner image URL
registrationOpen	TIMESTAMP	NOT NULL	Registration start
registrationClose	TIMESTAMP	NOT NULL	Registration end
status	ENUM	DEFAULT 'DRAFT'	Event status
paymentMethods	JSONB	NOT NULL	Enabled payment methods
bankDetails	JSONB	NULLABLE	Bank transfer info
cashInstructions	TEXT	NULLABLE	Cash payment instructions
createdAt	TIMESTAMP	DEFAULT NOW()	Creation time
updatedAt	TIMESTAMP	ON UPDATE NOW()	Last update

Status ENUM values: DRAFT, PUBLISHED, COMPLETED, CANCELLED

RaceCategories Table

Column	Type	Constraints	Description
id	UUID	PK	Primary key
eventId	UUID	FK → events.id, ON DELETE CASCADE	Parent event
name	VARCHAR(100)	NOT NULL	Category name (e.g., '5K')
distance	DECIMAL(10,2)	NULLABLE	Distance in kilometers

price	DECIMAL(10,2)	NOT NULL	Regular price
earlyBirdPrice	DECIMAL(10,2)	NULLABLE	Early bird price
earlyBirdDeadline	TIMESTAMP	NULLABLE	Early bird cutoff
slotLimit	INTEGER	NULLABLE	Max registrations
slotsTaken	INTEGER	DEFAULT 0	Current count
sortOrder	INTEGER	DEFAULT 0	Display order

Registrations Table

Column	Type	Constraints	Description
id	UUID	PK	Primary key
eventId	UUID	FK → events.id	Event reference
categoryId	UUID	FK → race_categories.id	Category reference
userId	UUID	FK → users.id, NULLABLE	Registered user (optional)
runnerName	VARCHAR(255)	NOT NULL	Runner full name
email	VARCHAR(255)	NOT NULL	Contact email
phone	VARCHAR(20)	NOT NULL	Contact phone
birthdate	DATE	NOT NULL	Date of birth
gender	ENUM	NOT NULL	MALE/FEMALE/OTHER
shirtSize	ENUM	NOT NULL	XS/S/M/L/XL/XXL
emergencyName	VARCHAR(255)	NOT NULL	Emergency contact
emergencyPhone	VARCHAR(20)	NOT NULL	Emergency phone
emergencyRelation	VARCHAR(100)	NOT NULL	Relationship
paymentMethod	ENUM	NOT NULL	Payment type
paymentStatus	ENUM	DEFAULT 'PENDING'	Payment state
proofUrl	VARCHAR(500)	NULLABLE	Payment proof image
amountPaid	DECIMAL(10,2)	NULLABLE	Amount received
bibNumber	VARCHAR(20)	NULLABLE	Assigned bib
confirmedBy	UUID	FK → users.id, NULLABLE	Staff who confirmed
confirmedAt	TIMESTAMP	NULLABLE	Confirmation time
notes	TEXT	NULLABLE	Internal notes
registeredAt	TIMESTAMP	DEFAULT NOW()	Registration time

PaymentMethod ENUM: QRPH, GCASH, MAYA, CARD, BANK_TRANSFER, CASH, FREE

PaymentStatus ENUM: PENDING, CONFIRMED, FAILED, REFUNDED, CANCELLED

Payments Table

Column	Type	Constraints	Description
id	UUID	PK	Primary key
registrationId	UUID	FK → registrations.id	Parent registration
paymongoPaymentId	VARCHAR(255)	NULLABLE, UNIQUE	PayMongo payment ID
paymongoCheckoutId	VARCHAR(255)	NULLABLE	Checkout session ID
amount	DECIMAL(10,2)	NOT NULL	Payment amount
currency	VARCHAR(3)	DEFAULT 'PHP'	Currency code
status	ENUM	NOT NULL	Payment status
paymentMethod	ENUM	NOT NULL	Method used
metadata	JSONB	NULLABLE	Additional data
paidAt	TIMESTAMP	NULLABLE	Payment timestamp
createdAt	TIMESTAMP	DEFAULT NOW()	Record creation

RefreshTokens Table

Column	Type	Constraints	Description
id	UUID	PK	Primary key
userId	UUID	FK → users.id, ON DELETE CASCADE	Token owner
token	VARCHAR(500)	UNIQUE, NOT NULL	Hashed token
expiresAt	TIMESTAMP	NOT NULL	Expiration time
createdAt	TIMESTAMP	DEFAULT NOW()	Creation time
revokedAt	TIMESTAMP	NULLABLE	Revocation time

Indexes

- users: email (unique), passwordResetToken
- organizations: slug (unique), ownerId
- events: slug (unique), organizationId, status, date
- registrations: eventId + email (unique), paymentStatus, registeredAt
- payments: registrationId, paymongoPaymentId (unique)
- refresh_tokens: userId, token (unique), expiresAt

API Specification

Base Configuration

- Base URL: /api/v1
- Content-Type: application/json
- Authentication: Bearer token in Authorization header
- Rate Limiting: 100 requests/minute for authenticated, 20/minute for anonymous

Response Format

All API responses follow a consistent structure:

Success Response: { data: T, meta?: { pagination } }

Error Response: { statusCode: number, message: string, errors?: ValidationError[] }

Authentication Endpoints

Method	Endpoint	Auth	Description
POST	/auth/register	No	Create new user account
POST	/auth/login	No	Authenticate and receive tokens
POST	/auth/refresh	Cookie	Refresh access token
POST	/auth/logout	Yes	Revoke refresh token
POST	/auth/forgot-password	No	Request password reset email
POST	/auth/reset-password	No	Reset password with token
POST	/auth/verify-email	No	Verify email with token
GET	/auth/me	Yes	Get current user profile

Users Endpoints

Method	Endpoint	Auth	Description
GET	/users	Admin	List all users (paginated)
GET	/users/:id	Admin	Get user by ID
PATCH	/users/:id	Owner/Admin	Update user profile
DELETE	/users/:id	Admin	Soft delete user

Organizations Endpoints

Method	Endpoint	Auth	Description
POST	/organizations	Yes	Create organization
GET	/organizations	Yes	List user's organizations
GET	/organizations/:slug	Yes	Get organization by slug
PATCH	/organizations/:id	Owner	Update organization
DELETE	/organizations/:id	Owner	Delete organization
POST	/organizations/:id/members	Owner	Invite member
DELETE	/organizations/:id/members/:userId	Owner	Remove member

Events Endpoints

Method	Endpoint	Auth	Description
GET	/events	No	List published events (public)
GET	/events/:slug	No	Get event details (public)
POST	/organizations/:orgId/events	Organizer	Create event
GET	/organizations/:orgId/events	Member	List org events
PATCH	/events/:id	Organizer	Update event
DELETE	/events/:id	Organizer	Delete event
POST	/events/:id/publish	Organizer	Publish event
POST	/events/:id/complete	Organizer	Mark as completed
POST	/events/:id/categories	Organizer	Add race category
PATCH	/events/:id/categories/:catId	Organizer	Update category
DELETE	/events/:id/categories/:catId	Organizer	Delete category

Registrations Endpoints

Method	Endpoint	Auth	Description
POST	/events/:eventId/register	Optional	Create registration
GET	/events/:eventId/registrations	Staff+	List registrations
GET	/registrations/:id	Owner/Staff	Get registration details
PATCH	/registrations/:id	Staff+	Update registration
POST	/registrations/:id/confirm	Staff+	Confirm payment
POST	/registrations/:id/cancel	Owner/Staff	Cancel registration
POST	/registrations/:id/upload-proof	Owner	Upload payment proof
GET	/events/:eventId/export	Organizer	Export to CSV/Excel

Payments Endpoints

Method	Endpoint	Auth	Description
POST	/payments/checkout	Optional	Create PayMongo checkout session

GET	/payments/:id	Owner/Staff	Get payment details
POST	/payments/webhook	No (verified)	PayMongo webhook handler

File Upload Endpoints

Method	Endpoint	Auth	Description
POST	/upload/image	Yes	Upload image (banner, proof)
DELETE	/upload/:key	Owner	Delete uploaded file

Frontend Architecture

Directory Structure

The frontend follows a feature-based organization pattern:

```
apps/web/src/
├── components/           # Shared UI components
│   ├── ui/              # shadcn/ui components
│   └── common/           # App-specific shared
├── features/             # Feature modules
│   ├── auth/            # Authentication
│   ├── events/          # Event management
│   ├── registrations/   # Registration flow
│   └── dashboard/       # Organizer dashboard
├── hooks/               # Custom React hooks
├── lib/                 # Utilities
│   ├── api.ts           # Axios instance
│   ├── utils.ts         # Helper functions
│   └── cn.ts            # Class name utility
├── routes/              # TanStack Router pages
├── stores/              # Zustand stores
└── styles/              # Global styles
```

Feature Module Structure

Each feature module contains:

- components/ - Feature-specific components
- hooks/ - Feature-specific hooks (useQuery, useMutation wrappers)
- api.ts - API functions for the feature
- types.ts - TypeScript interfaces
- schemas.ts - Zod validation schemas (or import from shared)

State Management Strategy

State Type	Solution	Example
Server State	TanStack Query	Event list, registrations, user data
Client State	Zustand	UI state, filters, modals
Form State	React Hook Form	Registration form, event creation
URL State	TanStack Router	Search params, pagination

Route Structure

Route	Component	Auth	Description
/	HomePage	No	Landing page
/events	EventsListPage	No	Public event listing
/events/:slug	EventDetailsPage	No	Event details + register
/events/:slug/register	RegistrationPage	No	Registration form
/login	LoginPage	No	User login
/register	RegisterPage	No	User registration
/dashboard	DashboardPage	Yes	Organizer dashboard
/dashboard/events	MyEventsPage	Yes	Manage events
/dashboard/events/new	CreateEventPage	Yes	Create event form
/dashboard/events/:id	EventDashboardPage	Yes	Event management
/dashboard/events/:id/registrations	RegistrationsPage	Yes	View registrations

Backend Architecture

Module Structure

NestJS modules follow domain-driven organization:

```
apps/api/src/
├── modules/
│   ├── auth/                # Authentication module
│   │   ├── auth.module.ts
│   │   ├── auth.controller.ts
│   │   ├── auth.service.ts
│   │   ├── strategies/      # Passport strategies
│   │   ├── guards/          # Auth guards
│   │   └── dto/              # Request/Response DTOs
│   ├── users/                # User management
│   ├── organizations/        # Organization management
│   ├── events/               # Event management
│   ├── registrations/        # Registration handling
│   ├── payments/              # Payment processing
│   ├── uploads/               # File upload handling
│   └── common/                # Shared module
├── config/                   # Configuration
├── prisma/                   # Prisma service
└── main.ts                   # Application entry
```

Module Components

Component	Responsibility	Example
Controller	HTTP request handling, routing	Handle POST /events
Service	Business logic, orchestration	Create event with categories
Repository	Data access (via Prisma)	Find events by org
DTO	Request/response validation	CreateEventDto
Guard	Authorization checks	RolesGuard
Interceptor	Request/response transformation	TransformInterceptor
Pipe	Validation, transformation	ValidationPipe

Error Handling

The API uses consistent HTTP status codes and error responses:

Status	Exception	Use Case
400	BadRequestException	Validation errors, malformed request
401	UnauthorizedException	Missing or invalid token
403	ForbiddenException	Insufficient permissions
404	NotFoundException	Resource not found
409	ConflictException	Duplicate resource, race condition
422	UnprocessableEntityException	Business rule violation
429	ThrottlerException	Rate limit exceeded
500	InternalServerErrorException	Unexpected server error

Job Queue Architecture

BullMQ handles async operations with Redis as the backing store:

Queue	Jobs	Priority
email	Send confirmation, password reset, reminders	Normal
payments	Process webhooks, retry failed payments	High
exports	Generate CSV/Excel reports	Low
notifications	Push notifications, SMS (Phase 2)	Normal

Payment Integration

PayMongo Integration

TakboHub integrates with PayMongo for online payments supporting GCash, Maya, and credit/debit cards.

Checkout Flow

9. Frontend calls POST /payments/checkout with registrationId
10. Backend creates PayMongo Checkout Session via API
11. Backend stores checkoutId in payments table
12. Backend returns checkout_url to frontend
13. Frontend redirects user to PayMongo checkout page
14. User completes payment on PayMongo
15. PayMongo redirects to success/failure URL
16. PayMongo sends webhook to /payments/webhook

Webhook Handling

- Verify webhook signature using PayMongo secret
- Parse event type: checkout_session.payment.paid, payment.failed
- Find registration by checkout_session.id
- Update payment status in database
- Update registration status
- Queue confirmation email job
- Return 200 OK to PayMongo

Idempotency

Webhook handlers must be idempotent to handle duplicate deliveries:

- Check if payment already processed before updating
- Use database transactions for consistency
- Log duplicate webhook events for monitoring

Manual Payment Confirmation

For bank transfer and cash payments, organizers manually confirm:

17. Registration created with status PENDING
18. Runner uploads proof (bank transfer) or pays cash
19. Organizer views pending payments in dashboard
20. Organizer reviews proof and clicks 'Confirm'
21. Backend updates status to CONFIRMED
22. Backend records confirmedBy and confirmedAt
23. Confirmation email sent to runner

Security Implementation

Authentication Security

- Passwords hashed with bcrypt (12 rounds)
- Access tokens expire in 15 minutes
- Refresh tokens expire in 7 days, stored as httpOnly cookie
- Refresh token rotation on each use
- All refresh tokens invalidated on password change
- Rate limiting on auth endpoints (5 attempts/minute)

API Security

- CORS configured for allowed origins only
- Helmet.js for security headers
- Request validation with class-validator
- SQL injection prevention via Prisma parameterized queries
- XSS prevention via response sanitization
- CSRF protection for state-changing operations

Data Security

- All traffic over HTTPS (enforced in production)
- Sensitive data encrypted at rest
- PII access logged for audit trail
- Database backups encrypted
- Environment variables for secrets (never committed)

File Upload Security

- File type validation (whitelist: jpg, png, webp, pdf)
- File size limits (5MB for images, 10MB for documents)
- Virus scanning (Phase 2)
- Unique filenames with UUID
- Signed URLs for private file access

Testing Strategy

Testing Pyramid

Level	Tools	Coverage Target	Focus
Unit	Jest, Vitest	80%	Services, utilities, hooks
Integration	Jest + Supertest	60%	API endpoints, database
E2E	Playwright	Critical paths	User flows

Backend Testing

- Unit tests for services with mocked dependencies
- Integration tests for controllers with test database
- Database seeding with `@faker-js/faker`
- Test containers for PostgreSQL and Redis
- Coverage reports with Jest

Frontend Testing

- Component tests with Testing Library
- Hook tests with `renderHook`
- MSW for API mocking
- Snapshot tests for UI components
- Accessibility testing with `axe-core`

Critical Test Scenarios

- User registration and login flow
- Event creation with categories
- Registration with online payment
- Registration with manual payment confirmation
- Payment webhook processing
- Registration export
- Permission checks for all protected routes

Development Workflow

Git Workflow

- main branch: production-ready code
- develop branch: integration branch
- Feature branches: feature/description
- Bug fix branches: fix/description
- Conventional commits: feat:, fix:, chore:, docs:

Code Quality

- ESLint for linting (flat config)
- Prettier for formatting
- TypeScript strict mode
- Husky for pre-commit hooks
- lint-staged for staged file checks

Local Development

24. Clone repository
25. Copy .env.example to .env
26. Run docker compose up -d (PostgreSQL, Redis, MinIO, Mailpit)
27. Run pnpm install
28. Run pnpm prisma migrate dev
29. Run pnpm prisma db seed
30. Run pnpm dev (starts both frontend and backend)
31. Access frontend at <http://localhost:5173>, API at <http://localhost:3000>

Environment Variables

Variable	Description	Example
DATABASE_URL	PostgreSQL connection string	postgresql://...
REDIS_URL	Redis connection string	redis://localhost:6379
JWT_SECRET	JWT signing secret	your-secret-key
JWT_REFRESH_SECRET	Refresh token secret	your-refresh-secret
PAYMONGO_SECRET_KEY	PayMongo API secret	sk_test_...
PAYMONGO_WEBHOOK_SECRET	Webhook verification	whsec_...
MINIO_ENDPOINT	S3 endpoint	localhost:9000
MINIO_ACCESS_KEY	S3 access key	minioadmin
MINIO_SECRET_KEY	S3 secret key	minioadmin

RESEND_API_KEY	Email service API key	re_...
FRONTEND_URL	Frontend URL for CORS	http://localhost:5173

Deployment

Production Environment

The application will be deployed to a VPS with Docker:

- Ubuntu 24.04 LTS
- Docker + Docker Compose
- Nginx as reverse proxy
- Let's Encrypt for SSL
- PostgreSQL 16 (managed or containerized)
- Redis 7 (managed or containerized)

Container Configuration

Service	Resources	Replicas
Frontend (Nginx)	256MB RAM, 0.25 CPU	1
API (NestJS)	512MB RAM, 0.5 CPU	1-2
Worker (BullMQ)	256MB RAM, 0.25 CPU	1
PostgreSQL	1GB RAM, 1 CPU	1
Redis	256MB RAM, 0.25 CPU	1

Monitoring

- Application logs aggregated with Pino
- Error tracking with Sentry (Phase 2)
- Uptime monitoring with UptimeRobot or similar
- Database monitoring with pgAdmin
- Queue monitoring with Bull Board

Backup Strategy

- Daily PostgreSQL backups (pg_dump)
- File storage synced to backup location
- 7-day retention for backups
- Tested restore procedure

— End of Document —